LOVELY
PROFESSIONAL
UNIVERSITY

Secure File management System

Hasamuddin Ansari
12320885
Roll.No. 59

Akash Ojha
12318785
Roll.No. 36

Prince Raj
12324706

Roll.No. 5

1. Introduction

In the digital age, managing and protecting sensitive data is a top priority for individuals and organizations alike. A Secure File Management System (SFMS), when implemented as an Operating Systems project, demonstrates key OS concepts such as file handling, access control, inter-process communication, and system security. This report outlines the architecture, features, technologies, and educational value of implementing a Secure File Management System as part of an OS course project. The project aims to simulate a real-world secure environment within the constraints and mechanisms provided by the operating system.

2. Objectives

- To implement core OS-level file handling operations securely.
- To demonstrate user authentication and access control mechanisms.
- To ensure data integrity, confidentiality, and proper file permissions.
- To enhance understanding of system calls, file systems, and process synchronization.
- To provide a user-friendly interface for file operations while maintaining strict security controls.

3. System Architecture

The Secure File Management System includes the following components, designed in line with OS-level principles:

- Command-Line Interface (CLI): User interface for uploading, downloading, and managing filesusing terminal commands.

- Authentication Module: Built using system-level programming (e.g., C with PAM or file-based loginvalidation).
- Access Control Layer: Implements Unix-style permissions (rwx) or Role-Based Access Control(RBAC).
- Encryption Engine: Utilizes libraries like OpenSSL to encrypt files during storage and decryptduring access.
- File Metadata Manager: Manages file ownership, permissions, timestamps, and change logs.- Storage Directory: Encrypted file repository on the local file system, with structured directory handling for organization.
- Process Synchronization Mechanisms: Ensures concurrent processes access files safely usingsemaphores or mutex locks.

4. Key Features

- System-Level Authentication: Uses user ID and passwords stored securely, or OS-level usermanagement.
- Access Permissions: Enforces read, write, execute permissions on a per-user or per-role basis.
- File Encryption: Ensures confidentiality using AES or RSA encryption with secure keymanagement.
- Audit Logging: Tracks all file activities in a system log file, including user actions and timestamps.
- Concurrent Access Handling: Prevents data corruption using process synchronization techniques.
- Backup and Restore: Supports manual and automated file backup options for disaster recovery.
- File Integrity Checking: Uses hashing (SHA-256) to detect unauthorized file modifications.

5. Technologies and Tools Used

- Programming Language: C / C++ / Python for system-level development.

- Operating System: Linux / Unix (preferably for direct system call usage).
- Encryption Library: OpenSSL or Python Cryptography module.
- File System Interface: POSIX-compliant file operations.
- Security: Manual implementation of RBAC, password hashing with SHA-256, file permission flags.
- Utilities: Shell scripting for automation, cron jobs for scheduled backups.

6. Educational Benefits

- Reinforces concepts like file systems, access control, and inter-process communication.
- Encourages secure programming practices and understanding of cryptographic functions.
- Provides hands-on experience with system-level APIs and OS internals.
- Enhances debugging and performance analysis skills in an OS environment.
- Prepares students for real-world system programming and security challenges.

7. Conclusion

Developing a Secure File Management System as an Operating Systems project is a valuable learning experience. It integrates various OS concepts into a real-world application, promoting better understanding of file operations, security mechanisms, and system-level programming. Such projects not only strengthen academic knowledge but also prepare students for practical challenges in cybersecurity and system development. Future enhancements could include graphical interfaces, integration with cloud storage, and advanced threat detection features.

8. References

1. Operating System Concepts by Silberschatz, Galvin, and Gagne

2. The Linux Programming Interface by Michael Kerrisk

3. OWASP Secure Coding Practices

4. NIST Cybersecurity Framework

5. ISO/IEC 27001 Information Security Standard

6. OpenSSL Project Documentation

7. POSIX Standard Documentation